Pandas I: Introduction to Data Manipulation

Wook-Shin Han

POSTECH

Spring 2026

Outline

1. Learning Objectives
2. Tabular Data
3. Why Pandas?
4. Data Structures
5. Working with Series
6. Working with DataFrames
7. Data Selection
8. Professional Development
9. Summary

Learning Objectives

By the end of this lecture, you will be able to:

- Understand the concept of tabular data and its importance in data science
- Master the three fundamental pandas data structures: Series, DataFrame, and Index
- Create DataFrames using multiple methods
- Perform data selection and slicing operations
- Apply professional data manipulation techniques

What is Tabular Data?

Definition

Tabular data is information organized in a table format with rows and columns, representing one of the most common and flexible ways to store data in data science.

Structure

- **Rows**: Each row represents one observation or instance
- **Columns**: Each column represents a feature or characteristic
- **Flexibility**: Easy to understand, manipulate, and analyze

Real-World Example: Elections Dataset

- Each row: A presidential candidate in a specific year
- Each column: Candidate information
  - Year
  - Name
  - Party
  - Popular Vote
  - Result
  - Percentage

Why Pandas?

Industry Standard

Pandas is the accepted standard tool for tabular data manipulation in both industry and academia.

Key Capabilities

- Arrange data in tabular format
- Extract information with specific conditions
- Perform data operations for insights
- Apply NumPy functions efficiently
- Execute vectorized computations for speed

Three Fundamental Data Structures

1. Series

1-dimensional labeled array data

- Think of it as a single column
- Contains values of the same type
- Has an associated index (labels)

2. DataFrame

2-dimensional tabular data

- Collection of Series sharing the same Index
- Rows and columns structure
- Primary working data structure

3. Index

Sequence of row/column labels

- Can be integers, strings, or other types
- Does not need to be unique
- Provides data alignment and selection capabilities

Series Anatomy: index, name, values

Create a Series with labeled index and a Series name

```
s = pd.Series([4, 2, 4, 6],
    index=['cat', 'penguin', 'dog', 'butterfly'],
    name='legs')
s.index.name = 'animal'
```

Inspect the components

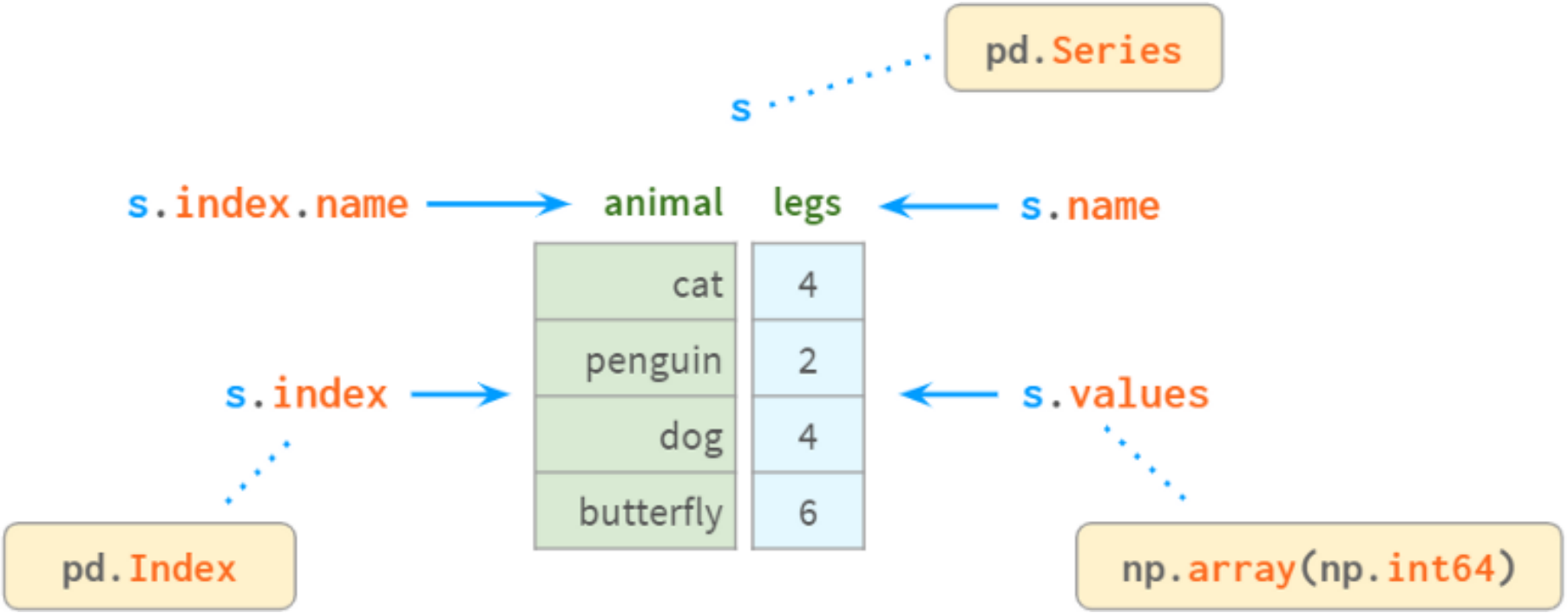
```
type(s)             # pandas.core.series.Series
s.index             # pd.Index([...], dtype='object', name='
    animal')
type(s.index)       # pandas.core.indexes.base.Index
s.index.name        # 'animal'  (label for the index axis)
s.name              # 'legs'    (label for the Series itself)
s.values            # np.ndarray([...], dtype=np.int64)
# Prefer s.to_numpy() for a consistent NumPy view across
    dtypes
```

- `s.index` holds the row labels (`pd.Index`); `s.index.name` is its title.
- `s.name` names the Series; useful after `reset_index()` or in joins.
- `s.values` returns a NumPy array; use `s.to_numpy()` when working with extension dtypes.

Working with Series — Core operations

- Series Index
- Series with Custom Index
- Series: Label vs. Position Indexing
- Boolean Filtering Idioms (`isin`, `between`)
- Take by Labels or Positions
- Index Edits: Assign, Sort, Reset, Delete
- Find Element by Value (Vector vs. Indexed Lookup)
- Missing Values: dtypes and `isna()`
- Missing Values II: Fill or Interpolate
- Missing Values III: Reductions with `NaN`
- Missing Values IV: Alignment in Arithmetic
- Sort, Rank, Top-k, Quantiles
- Cumulative, Difference, Lag, Return
- Where/Mask, Clip, Replace
- Map, Astype, To-NumPy
- Accessors: .str, .dt, .cat
- Windows: Rolling, Expanding, EWM
- Fast Scalars; Value Counts; Sampling

Series Index



- Index immutability: replace the whole index rather than editing in place, e.g. `s.index = pd.Index([...], name='animal')`.
- Check uniqueness: `s.index.is_unique`. If there are duplicates, `s.loc['label']` may return multiple rows.
- Types: values use `s.dtype`; labels use `s.index.dtype`. Mismatched dtypes often explain selection bugs.

Labeled Array Model vs Relational Model

- **Data abstraction.**
  - **Labeled array (pandas):** `pd.Series`/`pd.DataFrame` = arrays with *labels* (index, columns); order is meaningful and preserved.
  - **Relational (SQL):** relations (tables) as sets/bags of tuples with named attributes; logical order is irrelevant (order only via `ORDER BY`).
- **Identity & keys.**
  - Labeled array: `Index` provides labels (can be non-unique); behaves like an address space.
  - Relational: *keys* (PK/CK) identify tuples; constraints enforce uniqueness & referential integrity.
- **Missing data.**
  - Labeled array: `NaN`/`pd.NA`; many reductions use `skipna=True`.
  - Relational: `NULL` with three-valued logic; aggregates often ignore `NULL` by definition.
- **Core operations (mental mapping).**
  - `s1 + s2` (Series): *align by labels* then vectorized math ≈ `FULL OUTER JOIN` on keys then compute.
  - `df.merge` ≈ SQL `JOIN` (`inner`/`left`/`outer`/`cross`).
  - `df.groupby(...).agg(...)` ≈ SQL `GROUP BY` aggregates.
  - `pd.concat` (axis=0) ≈ `UNION ALL`; (axis=1) aligns on index like a join on row labels.

Setup

```
import pandas as pd
grades = pd.Series([85, 92, 78, 96],
                   index=['Alice', 'Bob', 'Charlie', 'Diana'
                       ],
                   name='grades')
```

- **Label-based:** `grades.loc[label / list / slice]`
  - Uses *index labels* (e.g., `'Alice'`).
  - Slices are *inclusive* of the stop label: `grades.loc['Alice':'Charlie']`.
  - Missing labels raise `KeyError`; use `reindex` if you want missing → `NaN`.
- **Position-based:** `grades.iloc[pos / list / slice]`
  - Uses *0..n-1* positions only.
  - Slices are *exclusive* of the stop: `grades.iloc[0:2]`.
  - Prefer `iloc` over `[]` when the index is integer-like to avoid ambiguity.
- **Boolean mask (conditional):** `grades[mask]` or `grades.loc[mask]`
  - `mask` is a same-length bool Series/array, e.g., `grades >= 90`.
  - Combine conditions with `&`, `|`, `~` and *parentheses*: `(grades >= 90) & (grades < 95)`.

Tip: For setting values, prefer `grades.loc[...]` on the left-hand side to avoid chained assignment.

Examples of Series Selection Methods

Setup

```
grades = pd.Series([85, 92, 78, 96],
                   index=['Alice', 'Bob', 'Charlie', 'Diana'
                       ],
                   name='grades')
```

Label-based: `grades.loc[label / list / slice]`

```
grades.loc['Alice']                 # 85
grades.loc[['Alice', 'Diana']]      # multiple labels
grades.loc['Alice':'Charlie']       # slice is INCLUSIVE of stop label
# grades.loc['Eve']                 # KeyError (use reindex to allow missing
    -> NaN)
```

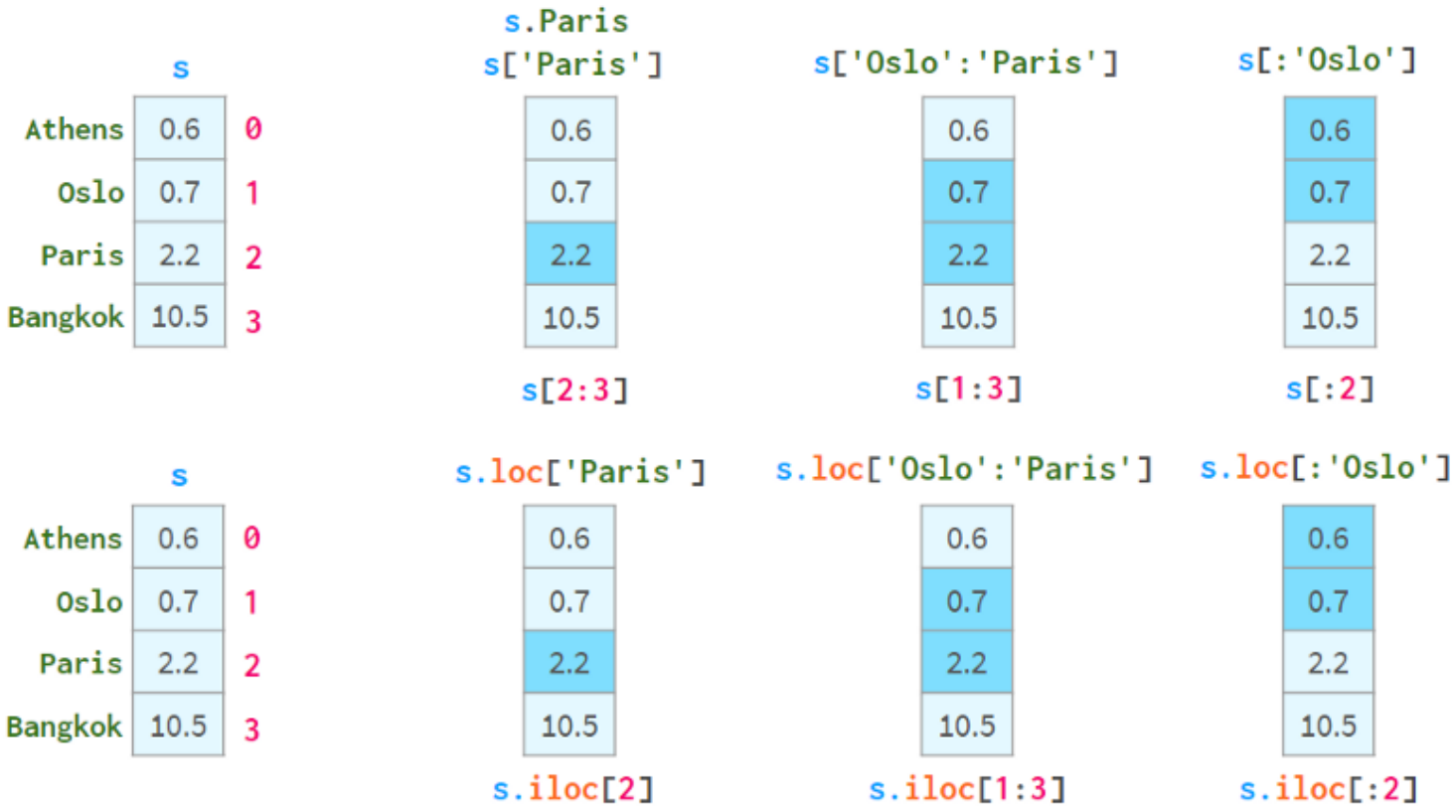
Position-based: `grades.iloc[pos / list / slice]`

```
grades.iloc[0]                      # 85
grades.iloc[[0, 3]]                 # positions list
grades.iloc[0:2]                    # slice is EXCLUSIVE of stop (Alice, Bob)
# Prefer iloc over [] when the index is integer-like (avoids ambiguity)
```

Boolean mask (conditional): `grades[mask]` **or** `grades.loc[mask]`

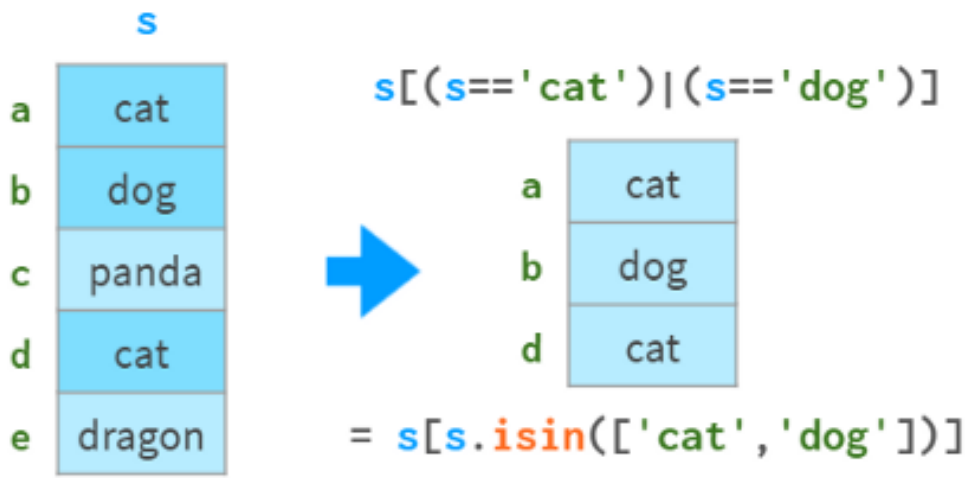
```
mask = grades >= 90                 # same-length boolean mask
grades[mask]                        # Bob, Diana
grades.loc[(grades >= 90) & (grades < 95)]   # combine with &, |, ~ and
    parentheses
```

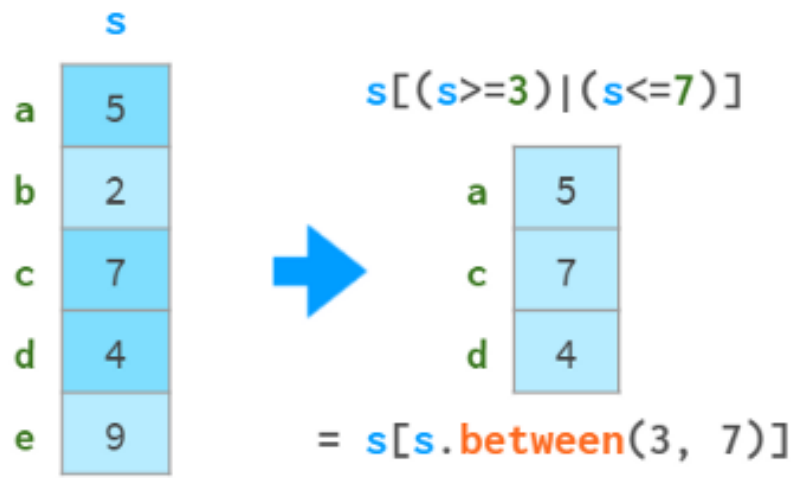
Series: Label vs. Position — why prefer `.loc`?



- **Explicitness:** `.loc` = labels, `.iloc` = positions. Plain `[]` is overloaded (`s['Paris']` label vs. `s[1:3]` positional slice).
- **Slice rule:** `loc` is *inclusive* of the stop; `iloc` is *exclusive*. Use `.loc`/`.iloc` to make intent clear.
- **No ambiguity with integers:** for integer-like indexes, `s[1]` means label 1. Use `.loc[1]` (label) or `.iloc[1]` (2nd element).

Boolean Mask on *Values* (`isin`, `between`)





- **Important:** `s == 'cat'` compares *values* element-wise, not labels. It yields a boolean mask the same length as `s`.
- Filter by *values*: `s[(s=='cat') | (s=='dog')]` ≡ `s[s.isin(['cat','dog'])]`.
- Filter by *labels* instead: `s.loc[s.index.isin(['a','d'])]` or `s.loc[s.index == 'a']`.
- Numeric ranges: use `&` (AND), not `|`; remember parentheses. `s[(s>=3) & (s<=7)]` ≡ `s.between(3, 7)` (inclusive by default).

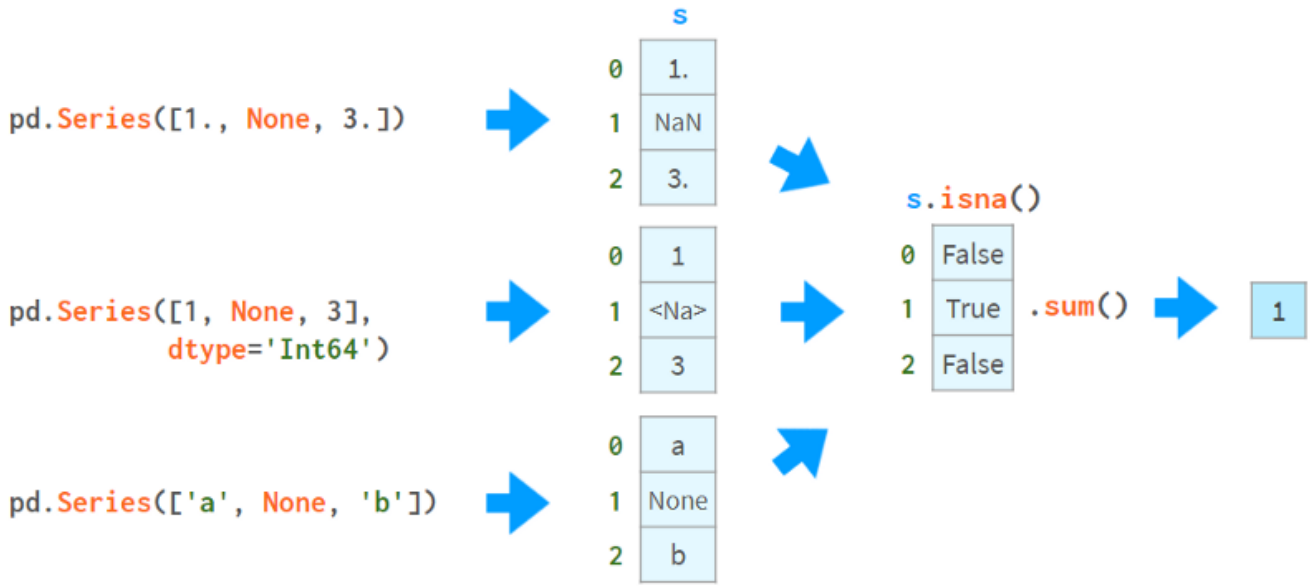
Find by Value — Vector scan vs Indexed lookup

- **Vector scan (values → labels/positions):**
  - `s.eq(v)`/`s==v` → mask, then `s.index[mask]` or `mask.to_numpy().nonzero()`.
  - Extremes: `s.idxmax()`/`s.idxmin()` return the *label* of max/min.
  - For ordered data, `s.searchsorted(v)` gives insertion position (on `s.values` or `s.sort_values()`).
- **Indexed lookup (keys → values):**
  - Normal case is `s.loc[key]`.
  - To map *value→label(s)* efficiently, build a reverse map once: `rev = pd.Series(s.index, index=s.values)` (handle duplicates separately).
- **Duplicates:** a value can occur multiple times; use group map to lists.
- **Pitfall:** value search is over *values*, not index labels; do not confuse with `.loc`.

Find by Value — Quick Examples

```
import pandas as pd
s = pd.Series([85, 92, 78, 92, 96], index=['Alice','Bob','Charlie','Eve','
    Diana'])

# 1) Vector scan: who scored 92?
who = s.index[s.eq(92)].tolist()          # ['Bob','Eve']

# 2) Label of max / min
top, low = s.idxmax(), s.idxmin()         # ('Diana', 'Charlie')

# 3) Reverse map (one-to-one only!)
rev = pd.Series(s.index, index=s.values)
rev.loc[96]                               # 'Diana'
# For duplicates -> list map:
rev_multi = s.reset_index().groupby(0)['index'].apply(list)
rev_multi.loc[92]                         # ['Bob','Eve']

# 4) Position(s) of a value
import numpy as np
pos = np.flatnonzero(s.to_numpy() == 92)  # array([1, 3])
```

Missing Values — dtypes and `isna()`

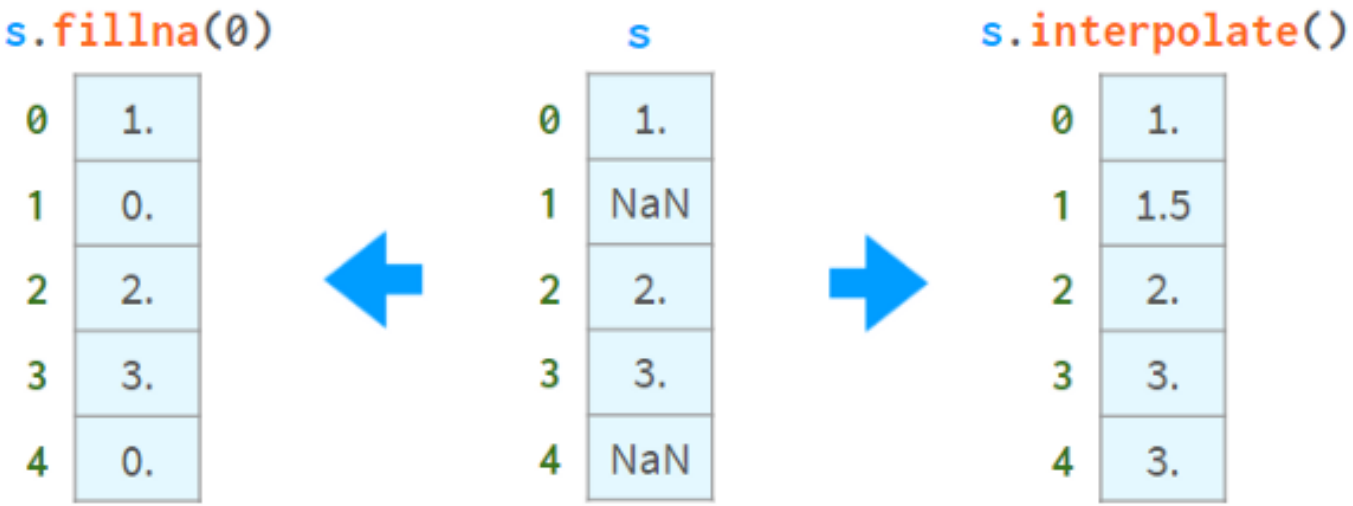


- Missing sentinels: `np.nan` (NumPy float NaN), `pd.NA` (pandas scalar for NA-aware dtypes).
- NA-aware dtypes: `Int64`, `Float64`, `boolean`, `string`; preserve type with missing via `pd.NA`. (i.e., three-valued logic)
- NumPy dtypes → upcasting: mixing ints with `np.nan` yields `float64`.
- Detect missing: `s.isna()` → `pd.Series[bool]`, aliases: `pd.isna`, `s.notna()`.
- Counting: `s.isna().sum()` (# missing), `s.count()` (# non-missing).
- Construction tips: use `dtype='Int64'` or `'boolean'` to avoid float upcast when you need integers/booleans with NA.

isna() and dtypes — Quick Examples

```
import numpy as np, pandas as pd

# Upcasting with NumPy NaN
s = pd.Series([1, 2, np.nan, 4])
s.dtype
# dtype('float64')

s.isna()
# 0    False
# 1    False
# 2     True
# 3    False
# dtype: bool

# NA-aware integer dtype keeps integers+NA
t = pd.Series([1, pd.NA, 3], dtype="Int64")
t, t.dtype
# 0       1
# 1    <NA>
# 2       3
# dtype: Int64

# DataFrame-wide missing map and counts
df = pd.DataFrame({"a":[1, pd.NA, 3], "b":[np.nan, 2.0, 3.0]})
df.isna()
df.isna().sum()       # per column
df.notna().all()      # no missing in a column?
```

Missing Values II — Fill or Interpolate

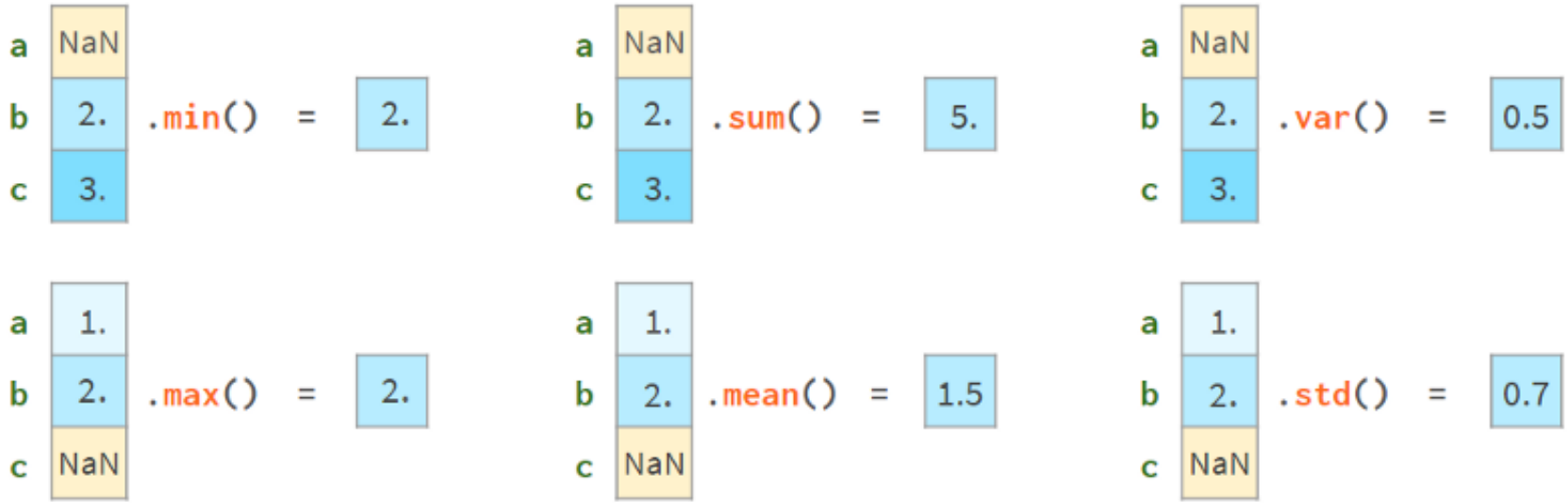


- **Direct fill:** `s.fillna(value)`; `df.fillna({col: val})`. Methods: `method='ffill'|'bfill'`, with `limit=` to cap consecutive fills.
- **Propagate last/next:** `s.ffill()` fill missing values with the last seen valid value (forward carry), `s.bfill()` (backward carry).
- **Interpolation:** `s.interpolate(method='linear'|'index'|'time', limit_direction=...)`. Requires monotonic index for `'index'`/`'time'` (DatetimeIndex for `'time'`).
- **When to use what:** constants/sentinels ⇒ `fillna`; carry last valid ⇒ `ffill/bfill`; trend-aware gaps ⇒ `interpolate`.
- **Caution:** Interpolation estimates values; document your choice of `method`, `limit`, and whether time spacing is uniform.

fillna, ffill/bfill, interpolate — Quick Examples

```
import pandas as pd, numpy as np

s = pd.Series([1.0, np.nan, np.nan, 4.0, np.nan])

s.fillna(0)
# 1.0, 0.0, 0.0, 4.0, 0.0

s.ffill()
# 1.0, 1.0, 1.0, 4.0, 4.0

s.bfill(limit=1)
# 1.0, nan, 4.0, 4.0, nan   # only first gap filled due to limit=1

# Time-aware interpolation
idx = pd.date_range("2024-01-01", periods=5, freq="D")
t = pd.Series([1.0, np.nan, 4.0, np.nan, 9.0], index=idx)
t.interpolate(method="time")
# linearly interpolates by actual time spacing

# Per-column fills with a dict
df = pd.DataFrame({"a":[1, np.nan, 3], "b":[np.nan, 2.5, np.nan]})
df.fillna({"a": 0, "b": df["b"].mean()})
```

Missing Values III — Reductions with NaN

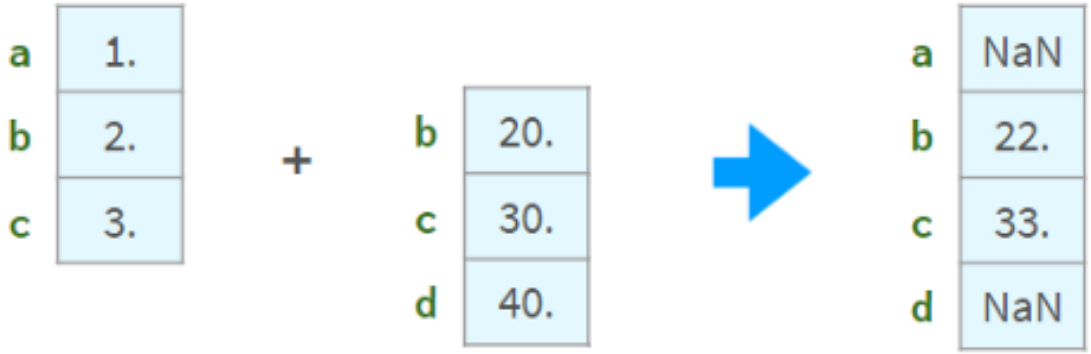


- NA-skipping by default: `s.sum()/mean()/min()/max()` use `skipna=True` (default). Control via `skipna=False` to require complete data.
- All-missing behavior: `s.mean()` → `NaN`; `s.sum()` → `0` (for numeric). `s.count()` counts non-missing elements.
- Robust counts: `s.isna().sum()` (# missing)
- Uniques with NA: `s.nunique(dropna=True)` ignores missing; set `dropna=False` to include.
- Boolean reductions: `s.any()` / `s.all()` ignore missing with `skipna=True`; use `skipna=False` to propagate `NaN`.

Reductions with NaN — Quick Examples

```
import numpy as np, pandas as pd

s = pd.Series([1.0, np.nan, 3.0, np.nan])

s.sum(), s.mean(), s.count()
# (4.0, 2.0, 2)      # sum skips NaN; mean over non-missing; count=2

s.mean(skipna=False)
# nan                # requires complete data

pd.Series([np.nan, np.nan]).sum(), pd.Series([np.nan, np.nan]).mean()
# (0.0, nan)

df = pd.DataFrame({"a":[1, np.nan, 3], "b":[2, 4, np.nan]})
df.mean(axis=0), df.mean(axis=1)
# a->2.0, b->3.0     # by column
# row means with NaNs skipped

# Boolean reductions
b = pd.Series([True, pd.NA, False], dtype="boolean")
b.any(), b.all()
# (True, False)      # NA ignored under skipna=True
```

Missing Values IV — Alignment in Arithmetic



- **Index alignment:** binary ops align on labels; union of keys is used. Non-overlapping labels → `NaN` in result.
- **Method forms with fill:** `s1.add(s2, fill_value=0)`, `df1.sub(df2, fill_value=0)`, `mul/div/pow` analogous.
- **DataFrame alignment:** aligns on both index and columns. Use `df1.add(df2, fill_value=0)` to avoid `NaN` where only one side has data.
- **Broadcasting with alignment:** `df.add(s, axis='index'|'columns')` matches along the given axis labels.
- **Preparation:** `reindex` to a common index/columns (optionally with `fill_value`) before arithmetic for deterministic outcomes.

Example of Alignment in Arithmetic

Code

```python
import pandas as pd

s1 = pd.Series({"a": 1.0, "b": 2.0})
s2 = pd.Series({"b": 10.0, "c": 100.0})

s1 + s2
s1.add(s2, fill_value=0)
```

Output

```
# s1 + s2
# a      NaN
# b     12.0
# c      NaN
# dtype: float64

# s1.add(s2, fill_value=0)
# a       1.0
# b      12.0
# c     100.0
# dtype: float64
```

Sort, Rank, Top-k, Quantiles

- `s.sort_values(ascending=...)` → `pd.Series`: reorder by *values*; stable within ties; preserves index.
- `s.sort_index(ascending=...)` → `pd.Series`: reorder by *labels* (index); stable; preserves index.
- `s.rank(method='dense', ascending=...)` → `pd.Series[float]`: assign ranks; `dense` gives 1,2,3,... even with ties.
- `s.nlargest(k)` → `pd.Series` / `s.nsmallest(k)` → `pd.Series`: fastest top-k/bottom-k by values (returned Series is sorted).
- `s.quantile(q)` → `float` | `pd.Series`: scalar if q is a single float in [0,1]; Series (indexed by q) if list-like.

Setup for examples

```
import pandas as pd
s = pd.Series([5, 2, 7, 4, 9], index=list('abcde'))
# a:5, b:2, c:7, d:4, e:9
```

Examples of Sort, Rank, Top-k, Quantiles

```
s.sort_values(ascending=False)
# e     9
# c     7
# a     5
# ...

s.sort_index()
# a     5
# b     2
# c     7
# ...

s.rank(method='dense', ascending=False)
# a     3.0
# b     5.0
# ...
# e     1.0

s.nlargest(2)
# e     9
# c     7

s.nsmallest(2)

s.quantile([0.25, 0.50, 0.75])
# 0.25    4.0
# 0.50    5.0
# 0.75    7.0
```

Cumulative, Difference, Lag, Return

- `s.cumsum()` $\rightarrow$ `pd.Series`: cumulative sum.
- `s.cummax()` $\rightarrow$ `pd.Series`: running maximum.
- `s.cummin()` $\rightarrow$ `pd.Series`: running minimum.
- `s.cumprod()` $\rightarrow$ `pd.Series`: cumulative product.
- `s.shift(periods=k, fill_value=None)` $\rightarrow$ `pd.Series`: **move values** by k rows while keeping the same index (lag if $k > 0$, lead if $k < 0$); first/last $|k|$ positions become `NaN` unless `fill_value` is given.
- `s.diff(periods=k)` $\rightarrow$ `pd.Series[float]`: **change vs. k-step lag**; exactly $s - s.\text{shift}(k)$. First k values are `NaN`; negative k computes a forward difference.
- `s.pct_change(periods=k)` $\rightarrow$ `pd.Series[float]`: **relative change** vs. k-step lag; exactly $\frac{s}{s.\text{shift}(k)} - 1$. First k values are `NaN`; beware division by zero $\rightarrow$ `inf`/`NaN`.

diff, shift, pct_change — Quick Examples

```
s = pd.Series([100, 110, 121], index=['t0','t1','t2'])

s.shift(1)                    # move values down by 1 (lag)
# t0      NaN
# t1    100.0
# t2    110.0

s.diff(1)                     # == s - s.shift(1)
# t0     NaN
# t1    10.0      (110 - 100)
# t2    11.0      (121 - 110)

s.pct_change(1)               # == s / s.shift(1) - 1
# t0       NaN
# t1      0.10     (110/100 - 1)
# t2      0.10     (121/110 - 1)

s.diff(2)                     # difference vs. two steps back
# t0     NaN
# t1     NaN
# t2    21.0       (121 - 100)

s.shift(-1)                   # lead: pull next value up
# t0    110.0
# t1    121.0
# t2      NaN
```

Tips:

- Use shift(k, fill_value=...) to avoid leading/trailing NaN

Where/Mask, Clip, Replace — What they do

- `s.where(cond, other)` → `pd.Series`: keep values where `cond`, else `other`.
- `s.mask(cond, other)` → `pd.Series`: inverse of `where`.
- `s.clip(lower, upper)` → `pd.Series`: clamp to bounds.
- `s.replace(to_replace, value)` → `pd.Series`: value remapping.

Where/Mask, Clip, Replace — Two-Column Examples

Code (s setup + ops)

```python
# Set up the Series s
s = pd.Series([5, -2, 7, -1, 9],
              index=list("abcde"))

# Where: keep >= 0, else 0
s_where = s.where(s >= 0, 0)

# Mask: set < 0 to 0 (inverse of where)
s_mask = s.mask(s < 0, 0)

# Clip: bound values to [0, 6]
s_clip = s.clip(lower=0, upper=6)

# Replace specific values
s_repl = s.replace({-2: 0, -1: 0})
```

Output

```
s_where
a    5
b    0
c    7
d    0
e    9

s_mask
a    5
b    0
c    7
d    0
e    9

s_clip
a    5
b    0
c    6
d    0
e    6

s_repl
a    5
b    0
c    7
d    0
e    9
```

Map, Astype, To-NumPy — What they do

- `s.map(func | dict)` → `pd.Series`: elementwise transform or dictionary lookup (index preserved). Unmatched keys become `NaN`; use `fillna(...)` if needed.
- `s.astype(dtype)` → `pd.Series`: cast values to a new dtype (e.g., `'float64'`, `'Int64'`, `'string'`). Fails if coercion is impossible; use `errors='ignore'` to skip.
- `s.to_numpy(copy=False)` → `np.ndarray`: NumPy view/array of the values. Prefer over `s.values` (works consistently with extension dtypes).

Map, Astype, To-NumPy — Two-Column Examples

Code

```
import pandas as pd

code2name = {'US':'United States', 'KR'
    'Korea'}
country = pd.Series(['US','KR','XX'])
country_name = country.map(code2name).
    fillna('Unknown')

s = pd.Series([5, 2, 7], index=list('abc
    ')).astype('float64')
a = s.to_numpy(copy=False)
```

Output

```
country_name
# 0    United States
# 1            Korea
# 2          Unknown
# dtype: object

s
# a    5.0
# b    2.0
# c    7.0
# dtype: float64

a
# array([5., 2., 7.])
```

Accessors: .str, .dt, .cat — What they do

- `s.str.method(...)` → `pd.Series[string|object]`: vectorized string ops (normalize, split, extract). Requires string/nullable string dtype for best behavior.
- `s.dt.method(...)` → `pd.Series`: datetime parts (`year`, `month`), conversions (`to_period`), floor/ceil/round. Requires datetime dtype.
- `s.astype('category').cat.method(...)` → `pd.Series | Categorical`: add/remove categories, order, encode. Categories are fixed—add with `add_categories` before assignment.

Accessors: .str, .dt, .cat — Two-Column Examples

Code

```python
import pandas as pd

email = pd.Series(['A@x.com','B@Y.com',
    None], dtype='string')
domain = email.str.strip().str.lower().
    str.split('@').str[-1]

dt = pd.to_datetime(pd.Series(['
    2025-09-01','2025-09-12']))
years   = dt.dt.year
periods = dt.dt.to_period('M')

cat = pd.Series(['a','b','a'], dtype='
    category')
cat  = cat.cat.add_categories(['c'])
    # add an unused 'c'
cat2 = cat.cat.remove_unused_categories
    ()
```

Output

```
domain
# 0     x.com
# 1     y.com
# 2      <NA>
# dtype: string

years
# 0     2025
# 1     2025
# dtype: int64

periods
# 0     2025-09
# 1     2025-09
# dtype: period[M]

cat2
# 0     a
# 1     b
# 2     a
# dtype: category
# Categories (2, object): ['a', 'b']
```

Windows: Rolling, Expanding, EWM — What they do (and why those numbers?)

Given $s = [5, 2, 7, 4, 9]$ with index a, b, c, d, e

- **Rolling mean** `s.rolling(w, min_periods).mean()` → `pd.Series[float]` Fixed-size window of length w. At each index t, average the last w points (fewer if #obs < `min_periods`). $\mathrm{rollMean}_w[t] = \frac{1}{n_t} \sum_{i=t-n_t+1}^{t} s[i]$ where $n_t = \min(w, \#\text{seen})$.
  *Examples (*w=3, `min_periods=1`*):* $b : \frac{5+2}{2} = 3.5,\ c : \frac{5+2+7}{3} = 4.6667,\ \cdots$
- **Expanding sum** `s.expanding().sum()` → `pd.Series` Growing window from the start (cumulative). $\mathrm{expandSum}[t] = \sum_{i \le t} s[i]$. *Examples:*
  $c : 5 + 2 + 7 = 14,\ e : 5 + 2 + 7 + 4 + 9 = 27.$
- **EWMA (adjust=False)** `s.ewm(alpha, adjust=False).mean()` → `pd.Series[float]` Exponentially weighted moving average (more weight on recent). $y_t = \alpha\, x_t + (1 - \alpha)\, y_{t-1}$ with $y_0 = x_0$. *For* $\alpha = 0.5$*:* $a : 5.0000$ (seed), $b : 0.5 \cdot 2 + 0.5 \cdot 5 = 3.5000,\ c : 0.5 \cdot 7 + 0.5 \cdot 3.5 = 5.2500,$ $d : 0.5 \cdot 4 + 0.5 \cdot 5.25 = 4.6250,\ e : 0.5 \cdot 9 + 0.5 \cdot 4.625 = 6.8125.$

Notes: `center=True` centers the rolling window; `min_periods` controls early `NaN`s; α can be given via `span/com/halflife`.

Example: Windows: Rolling, Expanding, EWM

Code

```
import pandas as pd
s = pd.Series([5, 2, 7, 4, 9], index=
    list('abcde'))

roll3 = s.rolling(3, min_periods=1).mean
    ().round(4)
expd  = s.expanding().sum()
expo  = s.ewm(alpha=0.5, adjust=False).
    mean().round(4)
```

Output (with why)

```
roll3
# a  5.0000   # (5)/1
# b  3.5000   # (5+2)/2
# c  4.6667   # (5+2+7)/3
# d  4.3333   # (2+7+4)/3
# e  6.6667   # (7+4+9)/3

expd
# a   5       # 5
# b   7       # 5+2
# c  14       # 5+2+7
# d  18       # 5+2+7+4
# e  27       # 5+2+7+4+9

expo
# a  5.0000   # seed: y0 = x0 = 5
# b  3.5000   # 0.5*2 + 0.5*5
# c  5.2500   # 0.5*7 + 0.5*3.5
# d  4.6250   # 0.5*4 + 0.5*5.25
# e  6.8125   # 0.5*9 + 0.5*4.625
```

Fast Scalars; Value Counts; Sampling — What they do

- `s.at[label]` → `scalar`: fast scalar by label (index lookup); raises `KeyError` if missing.
- `s.iat[pos]` → `scalar`: fast scalar by position (0-based); raises `IndexError` if out of bounds.
- `s.value_counts(dropna=..., normalize=...)` → `pd.Series`: frequency table (or proportions with `normalize=True`); `dropna=False` to count missing.
- `s.sample(n | frac, random_state)` → `pd.Series`: random rows (without replacement by default). Use `replace=True` for bootstrap-like sampling.

Fast Scalars; Value Counts; Sampling — Two-Column Examples

Code

```
import pandas as pd

s    = pd.Series([5, 2, 7, 4, 9], index=
     list('abcde'))
vals = pd.Series(['x','x','y', None, 'x'
     ])

c = s.at['c']; p = s.iat[2]
counts = vals.value_counts(dropna=False)
props  = vals.value_counts(normalize=
     True, dropna=False).round(2)
one    = s.sample(n=1, random_state=0)
```

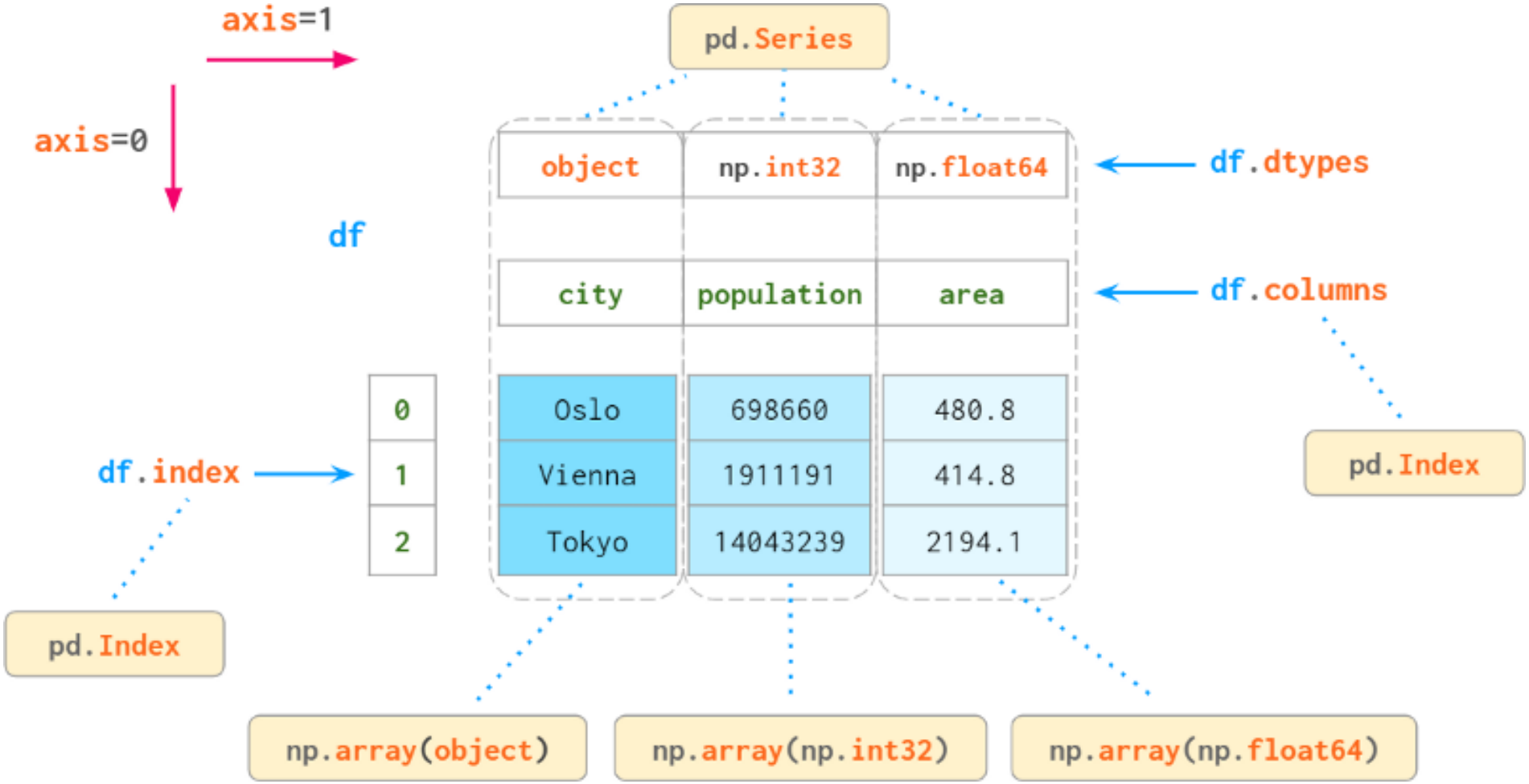
Output

```
(c, p)
# (7, 7)

counts
# x       3
# y       1
# NaN     1
# dtype: int64

props
# x    0.60
# y    0.20
# NaN  0.20
# dtype: float64

one
# e    9
# dtype: int64
```

Working with DataFrames — Core operations

- Anatomy & Diagnostics: `df.index`, `df.columns`, `df.dtypes`, `df.info()`, `df.select_dtypes(...)`
- Column Picks: `df['col']` (Series) vs. `df[['col']]` (DataFrame); `df.assign(new=...)`
- Label vs. Position Indexing: `df.loc[row_labels, col_labels]` (inclusive slices), `df.iloc[row_pos, col_pos]` (exclusive)
- Fast Scalars: `df.at[row, col]`, `df.iat[i, j]`
- Boolean Filtering: masks with `&`, `|`, `~`; `.isin()`, `.between()`, `.query(...)`
- Across-Columns Tests: `df.isna().any(axis=1)`, `df.gt(0).all(axis=1)`
- Safe Assignment: use `.loc[mask, 'col']` on LHS; `.copy()` after subsetting; align with `reindex`
- Missing Values: `df.isna()`, `fillna({col:val})`, `ffill/bfill` (axis=0/1), `interpolate`
- Alignment & Broadcasting: binary ops align on both axes; `df.add/ sub/ mul(..., fill_value=0)`; `add(series, axis='index'|'columns')`
- Sorting & Top-k: `df.sort_values(by=[...])`, `df.sort_index()`, `df.nlargest(k, 'col')/nsmallest`
- Reshape (Index/Columns): `set_index/reset_index`, `pivot/pivot_table`, `melt` (wide↔long)

DataFrame Anatomy: axes, columns, dtypes, index



- **Axes:** `axis=0` (rows), `axis=1` (columns).
- **Labels:** `df.columns` (column labels), `df.index` (row labels, a `pd.Index`).
- **Types:** each column is a `pd.Series` with its own `dtype`; see `df.dtypes`.
- Conversions to NumPy: `df['col'].to_numpy()` or `df.to_numpy()`.

DataFrame Anatomy — Python code example

```
# Recreate the example table with explicit dtypes
df = pd.DataFrame({
    "city":       pd.Series(["Oslo", "Vienna", "Tokyo"], dtype="object"),
    "population": pd.Series([698660, 1911191, 14043239], dtype=np.int32),
    "area":       pd.Series([480.8, 414.8, 2194.1],     dtype=np.float64),
})

# Inspect axes (axis=0 rows, axis=1 columns) and labels
print("index (axis=0)   :", df.index)      # RangeIndex(0, 3)
print("columns (axis=1) :", df.columns)    # Index([...], dtype='object')

# Each column is a Series with its own dtype
print("\nColumn dtypes:\n", df.dtypes)

# Conversions to NumPy (1D for a Series, 2D for a DataFrame)
print("\ncity -> numpy:", df["city"].to_numpy())                # dtype=
    object
print("pop+area -> numpy:\n", df[["population","area"]].to_numpy()  #
    numeric 2D

# Simple axis demo on numeric columns
num = df[["population","area"]]
print("\nColumn-wise sum (axis=0):\n", num.sum(axis=0))
print("\nRow-wise sum (axis=1):\n", num.sum(axis=1))
```

DataFrame Creation Methods

From CSV File

```
# Reading elections dataset
elections = pd.read_csv("elections.csv")
print(elections.head(2))
#    Year          Candidate                Party  Popular vote Result
# 0  1824     Andrew Jackson  Democratic-Republican       151271   loss
# 1  1824  John Quincy Adams  Democratic-Republican       113142    win
```

From Dictionary

```
student_data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Math': [85, 92, 78],
    'Science': [88, 95, 82]
}
df = pd.DataFrame(student_data)
print(df)
#      Name  Math  Science
# 0   Alice    85       88
# 1     Bob    92       95
# 2 Charlie    78       82
```

Read CSV: header and data

cities.csv:

csv = comma-separated values

```
name, population, area
Oslo, 698660, 480.8
Vienna, 1911191, 414.8
Tokyo, 14043239, 2194.1
```

header

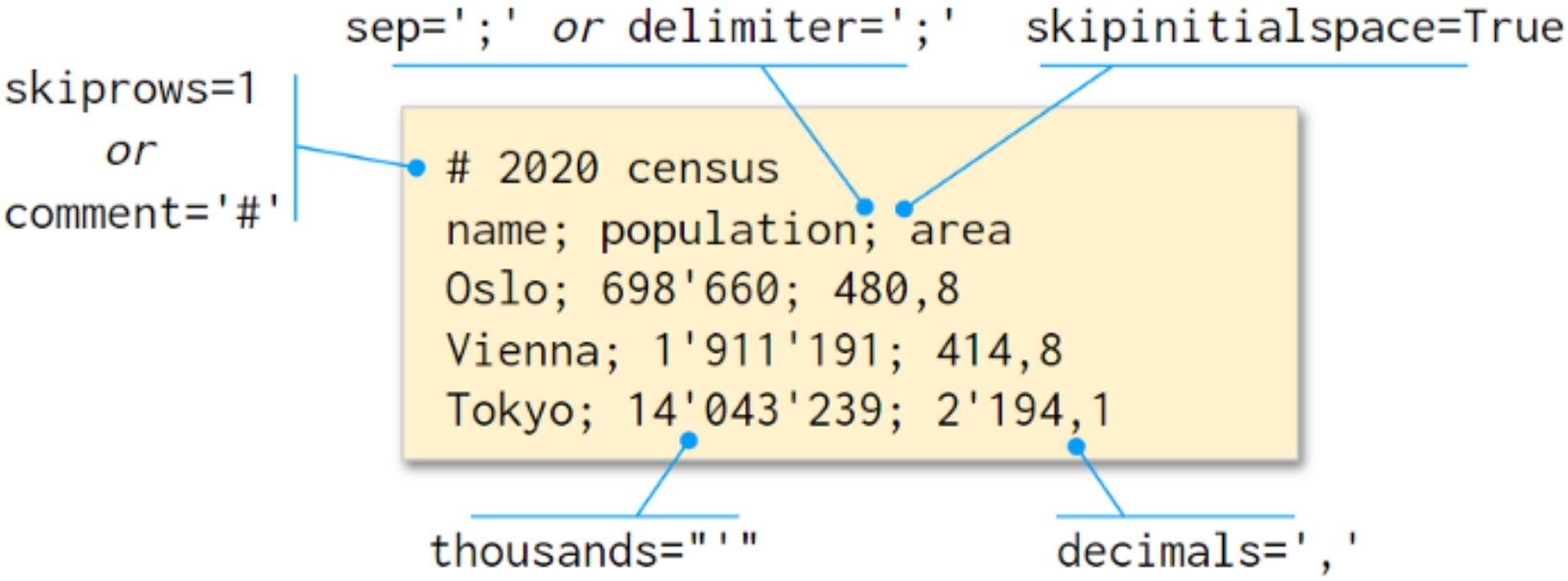
data

```
df = pd.read_csv('cities.csv')
```

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

- Basic load: `df = pd.read_csv('cities.csv')` (first row becomes header by default).
- Comma-separated values; pandas infers dtypes per column.
- Use `header=None` if the file has no header row.

CSV Parsing Options (locales, comments, spacing)



- Field separator: `sep=';'` (or `delimiter=';'`).
- European formats: `thousands="'"` (apostrophe), `decimal=','`.
- Ignore spaces after separators: `skipinitialspace=True`.
- Skip lines: `skiprows=1` or treat # as comments with `comment='#'`.

read_csv / to_csv: essential arguments

```
• pd.read_csv(
      filename,                 # build a dataframe out of this csv-file
      sep=',',                  # or delimiter=',' - field separator
      skipinitialspace=False,   # whether to allow spaces after sep
      quotechar='"',            # used for quoting strings that contain sep
      skiprows=n,               # skip the first n lines (default is 0)
      usecols=columns,          # only read those columns
      nrows=n,                  # only read n rows (useful for reading in chunks)
      index_col=columns,        # use column(s) as an index
      ...)                      # ~50 more less frequently used arguments

• df.to_csv(filename, ...)      # write data from the dataframe in a csv format
```

- `pd.read_csv(filename, sep=',', skipinitialspace=False, quotechar='"', skiprows=n, usecols=cols, nrows=n, index_col=cols, ...)`.
- Write out: `df.to_csv(filename, index=True/False, sep=',', na_rep='')`.
- Tip: use `usecols` to read a subset; `nrows` to sample; `index_col` to set the index on load.

Set the Index on Read vs. After Read

```
df = pd.read_csv('cities.csv')
```

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

```
df.set_index('name')
```

```
df = pd.read_csv('cities.csv',
        index_col='name')
```

name	population	area
Oslo	698660	480.8
Vienna	1911191	414.8
Tokyo	14043239	2194.1

- Two equivalent patterns: `df = pd.read_csv('cities.csv'); df = df.set_index('name')` vs. `df = pd.read_csv('cities.csv', index_col='name')`.
- Setting `index_col` at read-time avoids an extra copy.

More DataFrame Creation Examples

From Lists (2D)

```
# Creating DataFrame from nested lists
data = [['Alice', 85, 88], ['Bob', 92, 95], ['Charlie', 78, 82]]
df = pd.DataFrame(data, columns=['Name', 'Math', 'Science'])
```

From Series

```
# Combining Series into DataFrame
names = pd.Series(['Alice', 'Bob', 'Charlie'])
math_scores = pd.Series([85, 92, 78])
df = pd.DataFrame({'Name': names, 'Math': math_scores})
```

DataFrame Attributes with Examples

```
# Create sample DataFrame
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Math': [85, 92, 78],
    'Science': [88, 95, 82]
})

print("Index:", df.index)
# Index: RangeIndex(start=0, stop=3, step=1)

print("Columns:", df.columns)
# Columns: Index(['Name', 'Math', 'Science'], dtype='object')

print("Shape:", df.shape)
# Shape: (3, 3)

print("Data types:")
print(df.dtypes)
# Name        object
# Math         int64
# Science      int64
# dtype: object
```

Data Selection: Head and Tail

First n Rows

```
df.head(n)  # Default n=5
```

Last n Rows

```
df.tail(n)  # Default n=5
```

Use Cases

- Initial data exploration
- Verification after data operations
- Quick quality checks
- Presentation of sample data

Label-Based Selection: .loc Examples

```
# Sample data
df = elections.head()

# Single value selection
candidate = df.loc[0, 'Candidate']
print(candidate)  # Andrew Jackson

# Multiple rows, single column
candidates = df.loc[0:2, 'Candidate']
print(candidates)
# 0       Andrew Jackson
# 1    John Quincy Adams
# 2       Andrew Jackson
# Name: Candidate, dtype: object

# Multiple rows and columns (inclusive slicing)
subset = df.loc[0:2, 'Year':'Party']
print(subset)
#    Year         Candidate                Party
# 0  1824    Andrew Jackson  Democratic-Republican
# 1  1824  John Quincy Adams  Democratic-Republican
# 2  1828    Andrew Jackson            Democratic
```

More .loc Examples

```
# Select all rows, specific columns
winners = df.loc[:, ['Candidate', 'Result']]
print(winners)
#          Candidate Result
# 0    Andrew Jackson   loss
# 1  John Quincy Adams    win
# 2    Andrew Jackson    win

# Conditional selection with .loc
winners_only = df.loc[df['Result'] == 'win', ['Candidate', 'Year']]
print(winners_only)
#          Candidate  Year
# 1  John Quincy Adams  1824
# 2    Andrew Jackson  1828

# Boolean indexing
democratic = df.loc[df['Party'].str.contains('Democratic')]
print(democratic.shape)  # (3, 6)
```

Position-Based Selection: .iloc Examples

```
# Position-based selection (remember: exclusive slicing)
df = elections.head()

# Single value by position
first_candidate = df.iloc[0, 1]  # Row 0, Column 1
print(first_candidate)  # Andrew Jackson

# First 3 rows, first 4 columns (exclusive of endpoint)
subset = df.iloc[0:3, 0:4]
print(subset)
#    Year         Candidate                Party  Popular vote
# 0  1824    Andrew Jackson  Democratic-Republican      151271
# 1  1824  John Quincy Adams  Democratic-Republican      113142
# 2  1828    Andrew Jackson            Democratic      642806

# Last 2 rows, all columns
last_rows = df.iloc[-2:, :]
print(last_rows.shape)  # (2, 6)

# Every other row, specific columns
every_other = df.iloc[::2, [1, 4]]  # Columns 1 and 4
```

Context-Dependent Selection: []

Row Slicing

```
df[0:4]  # First four rows
```

Column Selection (List)

```
df[['Year', 'Candidate', 'Party']]  # Multiple columns
```

Single Column Selection

```
df['Candidate']  # Returns Series
```

Professional Usage

Most commonly used selector due to conciseness, especially for column selection operations.

Practical Example: Data Analysis Workflow

```
# Step 1: Load and inspect data
elections = pd.read_csv("elections.csv")
print(f"Dataset shape: {elections.shape}")
print(f"Columns: {list(elections.columns)}")

# Step 2: Basic exploration
print("First few rows:")
print(elections.head(3))

# Step 3: Specific analysis
print("Recent winners (2020-2024):")
recent_winners = elections[
    (elections['Year'] >= 2020) &
    (elections['Result'] == 'win')
][['Year', 'Candidate', 'Party']]
print(recent_winners)

# Step 4: Summary statistics
print(f"Total elections covered: {elections['Year'].nunique()}")
print(f"Unique candidates: {elections['Candidate'].nunique()}")
```

Indexing Methods Summary

Method	Use Case	Slicing	Input Type
`.loc`	Label-based selection	Inclusive	Labels
`.iloc`	Position-based selection	Exclusive	Integers
`[]`	Context-dependent	Varies	Mixed

Best Practices

Best Practices

- Use `.loc` for label-based operations (safer)
- Use `.iloc` for position-based logic
- Use `[]` for simple column selection
- Always consider data ordering when using `.iloc`

Professional Development Tips

Documentation Skills

- Pandas documentation is extensive
- Stack Overflow provides practical solutions
- Google search skills are essential
- Reading and implementing documentation is a core data science skill

Learning Approach

- Start with fundamental concepts
- Practice with real datasets
- Build familiarity through repetition
- Reference materials will be provided for assessments

Summary

Key Takeaways

1. **Tabular data** is fundamental to data science work
2. **Three core structures**: Series, DataFrame, Index
3. **Multiple creation methods** for different data sources
4. **Three selection approaches**: .loc, .iloc, []
5. **Professional practice** requires documentation skills

Next Steps

- Practice with real datasets
- Explore pandas documentation
- Prepare for Pandas II advanced operations
- Build comfort with data selection patterns

Questions and Discussion

Prepare for hands-on practice with real-world datasets in the next session